Kubernetes Practical & Troubleshooting Assessment

60 Scenario-Based Single-Choice Questions (With Detailed Answer Key)

Topics: Section 8 (Objects, Resources, Config, Policy, Security, Metadata) · Section 9 (Pod Architecture, Scheduling, Probes, Lifecycle) · Section 10 (Workload Controllers)

Instructions: Choose the **single best answer** for each question. Every question is based on real-world production scenarios, `kubectl` error traces, common misconceptions, and architectural trade-offs. Detailed answers and explanations are provided in the final section of this document.

Contents

1 Part 1: Kubernetes Objects, Resources, Config, Security & Metadata (Q1–Q24) 2

2 Part 2: Pod Concepts, Architecture, Scheduling, Probes & Lifecycle (Q25–Q42) 12

3 Part 3: Workload Management & Controllers (Q43–Q60) 19

4 Answer Key & Detailed Technical Explanations 26

1 Part 1: Kubernetes Objects, Resources, Config, Security & Metadata (Q1–Q24)

Question 1: API Resources vs. Objects & RBAC Troubleshooting

A developer runs a CI/CD pipeline using a ServiceAccount named `ci-bot` in the `staging` namespace. The pipeline fails with the following error trace:

```
Error from server (Forbidden): deployments.apps is forbidden: User
"system:serviceaccount:staging:ci-bot" cannot create resource "deployments"
in API group "apps" in the namespace "staging"
```

You inspect the existing `Role` bound to `ci-bot`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: staging
  name: deployment-creator
rules:
- apiGroups: [""]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
```

Why is the pipeline still receiving a `Forbidden` error?

A. A `Role` cannot grant permissions on `deployments`; you must use a `ClusterRole` and `ClusterRoleBinding`

B. The `apiGroups` field is set to `[""]` (the core API group), whereas the `deployments` resource belongs to the `"apps"` API group.

C. The `resources` field in RBAC must use the capitalized Kind name `["Deployment"]` instead of the lowercase plural resource endpoint `["deployments"]`.

D. ServiceAccounts cannot authenticate to the API server unless a `Secret` of type `Opaque` is manually attached to the `RoleBinding`.

Question 2: Custom Resources (CRD) Misconception

A junior engineer applies a `CustomResourceDefinition` (CRD) named `backups.devopscube.com` to the cluster and then successfully creates a custom object `kind: Backup` named `nightly-etcd-backup`. Running `kubectl get backups` returns the object, and it is persisted in `etcd`. However, no actual etcd backup is ever executed or uploaded to S3. What is the root cause?

A. Custom Resources are not stored in `etcd` unless the `--enable-custom-storage` flag is passed to `kube-apiserver`.

B. A CRD only extends the Kubernetes API to store and validate the declarative state; a separate Custom Controller (Operator) must be running to watch the resource and execute the backend backup logic.

C. Custom Resources only work in the `kube-system` namespace by default.

D. You cannot use `kubectl` with Custom Resources; you must send raw HTTP POST requests directly to `/registry/backups`.

Question 3: Object Name vs. UID Uniqueness

An engineer tries to apply a manifest containing two objects in the `production` namespace: a `Pod` named `payment-api` and a `Service` named `payment-api`. Later, they delete the `Pod` named `payment-api` and immediately recreate it from the exact same YAML file. Which statement accurately describes how Kubernetes handles object names and UIDs in this scenario?

A. The apply fails because two objects of different Kinds (`Pod` and `Service`) cannot share the name `payment-api` in the same namespace.

B. Both the `Pod` and `Service` can share the name `payment-api`, and when the Pod is recreated from the same YAML, it retains its original `metadata.uid`.

C. Both the `Pod` and `Service` can share the name `payment-api` because they are different Kinds, and when the Pod is deleted and recreated, Kubernetes assigns a brand-new `metadata.uid` to the new Pod instance.

D. Kubernetes stores objects in `etcd` strictly by name without UIDs, overwriting the Service object when the Pod is created.

Question 4: Multi-Object Manifest File Limits

A DevOps team bundles all microservices for an environment into a single large YAML manifest file separated by `---` and applies it via `kubectl apply -f full-stack.yaml`. When the file grows to 2.5 MB, the API server rejects the request. Why?

A. A Kubernetes YAML manifest file can only contain a maximum of 5 objects separated by `---`.

B. While there is no hard limit on the number of objects in a manifest, Kubernetes enforces a default request/object size limit of roughly 1 MB (up to 1.5 MB in etcd) per manifest/object payload.

C. `kubectl apply` only reads the first object in a multi-document YAML file unless `--all-objects=true` is specified.

D. Combining `Service` and `Deployment` objects in the same YAML file is forbidden by the core API schema.

Question 5: ConfigMap Missing Key Trace (`CreateContainerConfigError`)

You deploy a Pod that references a specific key from a `ConfigMap` as an environment variable. Running `kubectl get pods` shows:

```
NAME              READY   STATUS                       RESTARTS   AGE
order-worker-pod  0/1     CreateContainerConfigError   0          25s
```

Running `kubectl describe pod order-worker-pod` reveals:

```
Warning  Failed  4s (x3 over 24s)  kubelet  Error: couldn't find key DB_PORT
in ConfigMap default/app-config
```

You verify that `app-config` exists, and `DB_PORT` is an optional setting that your application can run without. How can you prevent the Pod from failing to start if `DB_PORT` is absent in the ConfigMap, without adding the key to the ConfigMap?

A. Set `ignoreMissingKeys: true` in the `ConfigMap` metadata.

B. Add `optional: true` inside the Pod's `valueFrom.configMapKeyRef` block.

C. Change the Pod's `restartPolicy` to `OnFailure`.

D. Mount the ConfigMap using `hostPath` instead of `configMapKeyRef`.

Question 6: ConfigMap Live Update Misconception (`subPath` Gotcha)

A team updates a `ConfigMap` named `nginx-conf` using `kubectl edit configmap nginx-conf`. They expect their running Nginx Pods to automatically receive the updated configuration file within a minute (after the kubelet sync period) without restarting the Pods. However, 30 minutes later, the file inside the container still shows the old configuration. You inspect the Pod's volume mount:

```
volumeMounts:
- name: config-vol
  mountPath: /etc/nginx/nginx.conf
  subPath: nginx.conf
```

Why did the file inside the container not update automatically?

A. ConfigMaps mounted as volumes are never updated dynamically under any circumstances.

B. Containers using `subPath` volume mounts do not receive automatic ConfigMap or Secret updates when the underlying object changes; a Pod restart or full directory mount (without `subPath`) is required.

C. `kubectl edit` only modifies the local client cache and does not update `etcd`.

D. ConfigMap volume updates only work if `immutable: true` is set on the ConfigMap.

Question 7: Cross-Namespace Resource Isolation

An application Pod running in the `frontend` namespace needs to connect to a PostgreSQL Service located in the `database` namespace, and also wants to mount a `Secret` named `db-creds` that currently resides in the `database` namespace. Which statement is true?

A. The Pod can neither reach the Service across namespaces nor mount the Secret across namespaces.

B. The Pod can reach the Service via DNS (`postgres-svc.database.svc.cluster.local`) and can also mount the Secret directly by setting `secretName: database/db-creds`.

C. The Pod can reach the Service across namespaces via DNS (unless blocked by a NetworkPolicy), but it cannot mount a `Secret` or `ConfigMap` from another namespace; the Secret must exist in the `frontend` namespace.

D. Both the Service and the Secret are cluster-scoped resources and ignore namespaces altogether.

Question 8: Identifying Cluster-Scoped vs. Namespaced Resources

A cluster admin runs `kubectl delete namespace sandbox`. Which of the following resources will **NOT** be deleted by this operation because it is a cluster-scoped (non-namespaced) object?

A. `PersistentVolumeClaim` (PVC) created inside `sandbox`.

B. `PersistentVolume` (PV) and `StorageClass` objects in the cluster.

C. `ResourceQuota` and `LimitRange` policies applied to `sandbox`.

D. `ServiceAccount` and `RoleBinding` objects in `sandbox`.

Question 9: ResourceQuota vs. LimitRange Interaction Failure

A cluster administrator applies the following `ResourceQuota` to the `payments` namespace:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: payments
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
```

No `LimitRange` exists in the `payments` namespace. A developer deploys a standard Nginx Pod manifest that **does not specify** a `resources` (`requests` or `limits`) block. What happens when the Pod is submitted?

A. The Pod is scheduled normally and consumes whatever unbound CPU and memory are available on the worker node.

B. The API server rejects the Pod creation with `Error from server (Forbidden): failed quota: compute-quota: must specify limits.cpu,limits.memory,requests.cpu,requests.memory`.

C. The `ResourceQuota` controller automatically injects a default of `100m` CPU and `128Mi` memory into the Pod.

D. The Pod enters the `Pending` state indefinitely waiting for the scheduler to calculate its resource usage at runtime.

Question 10: LimitRange Validation vs. Existing Pods

In the `analytics` namespace, three Pods are currently running with `limits.memory: 4Gi`. An administrator then applies a new `LimitRange` object to the `analytics` namespace that sets `max.memory: 2Gi` per container. What happens to the three existing running Pods?

A. All three existing Pods are immediately terminated by the `LimitRange` admission controller.

B. Their memory limits are dynamically hot-resized down to `2Gi` without restarting the containers.

C. Nothing happens to the existing running Pods; `LimitRange` validations and defaults are only enforced at Pod admission time (when new Pods are created or updated).

D. The `LimitRange` creation fails with a conflict error until the existing Pods are manually deleted.

Question 11: PodDisruptionBudget (PDB) Blocking Node Drain

You need to patch the OS kernel on worker node `worker-02`. You execute `kubectl drain worker-02 --ignore-daemonsets`, but the command hangs repeatedly with the following output:

```
error when evicting pods/"auth-service-6d4cf56db-x9v2k" -n "prod" (will retry
after 5s): Cannot evict pod as it would violate the pod's disruption budget.
```

You inspect the Deployment and the PDB for `auth-service`:

```
Deployment replicas: 2 (currently 2/2 Running)
PDB spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: auth-service
```

Why is the drain blocked, and what is the misconception here?

A. `kubectl drain` is an involuntary disruption, so PDBs always block it completely.

B. Because the Deployment has `replicas: 2` and the PDB requires `minAvailable: 2`, the allowed disruptions count (`disruptionsAllowed`) is `0`; no voluntary eviction can ever proceed unless replicas are scaled up or `minAvailable` is lowered to `1`.

C. PDBs require at least 5 replicas to function properly.

D. You must pass `--delete-emptydir-data` to bypass a PodDisruptionBudget.

Question 12: PDB Behavior During Physical Node Hardware Crash

A Deployment has 3 replicas distributed across 3 nodes and a `PodDisruptionBudget` configured with `maxUnavailable: 1`. Suddenly, a power outage causes 2 worker nodes to crash simultaneously. How does the PDB affect this event?

A. The PDB prevents the second node from powering off until the first Pod is rescheduled.

B. The PDB has no effect on preventing hardware failures; PDBs only govern **voluntary disruptions** initiated via the Eviction API (such as `kubectl drain` or cluster autoscaler scale-down), not involuntary hardware crashes.

C. The PDB locks the Deployment controller so it cannot recreate the failed Pods on healthy nodes.

D. The API server deletes the PDB automatically when `maxUnavailable` is exceeded during an outage.

Question 13: Pod Priority and Preemption Mechanics

Your cluster nodes are at 95% CPU request capacity. A critical batch job Pod is submitted with a `priorityClassName: high-priority-critical` (value `1000000`). No single node has enough free CPU to schedule this Pod, but `worker-01` is running several Pods with `priorityClassName: low-priority-dev` (value `100`). What happens next?

A. The high-priority Pod fails immediately with Status `OutOfcpu`.

B. `kube-scheduler` triggers preemption: it gracefully evicts (terminates) one or more `low-priority-dev` Pods on `worker-01` so the `high-priority-critical` Pod can be scheduled there.

C. Kubernetes automatically steals CPU cycles from the low-priority Pods without terminating them by rewriting their `spec.resources.requests`.

D. Preemption only works if all Pods belong to the same ReplicaSet.

Question 14: Kubernetes Secrets Base64 Misconception

A security auditor reviews your cluster where developers store database passwords in standard Kubernetes `Secret` objects. The developers claim that because `kubectl get secret db-pass -o yaml` displays gibberish strings like `cGFzc3dvcmQxMjM=`, the passwords are encrypted by default in `etcd`. Why is the developers' claim incorrect?

A. Secret values in YAML manifests are merely **Base64-encoded** (not encrypted) and are stored in unencrypted plaintext inside `etcd` by default unless Encryption at Rest is explicitly configured on the API server.

B. Kubernetes uses MD5 one-way hashing for Secrets, meaning pods cannot decode them without an init container.

C. Secrets are stored on the worker node's root disk in `/var/log/secrets.txt` in plaintext.

D. Base64 strings can only be decoded by a `ClusterRole` administrator.

Question 15: Private Registry `ImagePullBackOff` & Secret Types

You deploy a Pod using a private container image hosted on `registry.company.internal/payments:v2`. The Pod stays in `ImagePullBackOff`, and `kubectl describe pod` shows:

```
Warning Failed 12s kubelet Failed to pull image "registry.company.internal/payments:v2": unauthorized: authentication required
```

You already created a Secret containing your registry credentials, but the Pod still cannot pull the image. Which combination is required to fix this?

A. Create a Secret of type `Opaque` and mount it at `/etc/docker/certs.d` inside the container.

B. Create a Secret of type `kubernetes.io/dockerconfigjson` (or via `kubectl create secret docker-registry`) and reference it under `spec.imagePullSecrets` in the Pod (or link it to the Pod's `ServiceAccount`).

C. Create a `ConfigMap` named `docker-login` and pass it via `envFrom`.

D. Add the registry username and password directly into `metadata.annotations` of the Pod.

Question 16: Hardening Pods with `automountServiceAccountToken`

A security scan flags that a public-facing stateless web server Pod has a valid Kubernetes API bearer token mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token`. The web application never communicates with the Kubernetes API server. What is the best practice to eliminate the risk of an attacker stealing this token if the web app is compromised?

A. Delete the `default` ServiceAccount from the namespace.

B. Set `automountServiceAccountToken: false` on the Pod spec (or on the ServiceAccount object).

C. Use an Init Container to run `rm -rf /var/run/secrets` before the main container starts.

D. Bind the `cluster-admin` ClusterRole to the ServiceAccount with `verbs: ["none"]`.

Question 17: Reusing a ClusterRole Across Namespaces

You want to grant read-only access to `Secrets` to three different teams (`team-a`, `team-b`, `team-c`), strictly isolating each team so they can **only** read `Secrets` within their own respective namespace (`ns-a`, `ns-b`, `ns-c`). You do not want to maintain three duplicate `Role` YAML definitions. What is the standard Kubernetes RBAC pattern for this?

A. Create one `ClusterRole` defining the `secrets` read rules, and create a separate `RoleBinding` inside each namespace (`ns-a`, `ns-b`, `ns-c`) referencing that single `ClusterRole`.

B. Create one `ClusterRole` and one `ClusterRoleBinding`, then add a `namespaceSelector` field inside the `ClusterRoleBinding` spec.

C. Create one `Role` in the `default` namespace and reference it using three `ClusterRoleBindings`.

D. It is impossible; referencing a `ClusterRole` always grants cluster-wide access across all namespaces.

Question 18: RBAC Subresources Troubleshooting (`pods/log` & `pods/exec`)

A support engineer is granted a `Role` in the `prod` namespace with the following rule:

```
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch", "create"]
```

The engineer can successfully run `kubectl get pods -n prod`, but when they run `kubectl logs my-pod -n prod` or `kubectl exec -it my-pod -n prod -- sh`, they receive a `Forbidden` error. Why?

A. Reading logs and executing commands require node-level SSH access, not RBAC permissions.

B. In Kubernetes RBAC, `pods/log` and `pods/exec` are separate API subresources that must be explicitly listed in the `resources` array (e.g., `resources: ["pods", "pods/log", "pods/exec"]`).

C. The `logs` and `exec` endpoints belong to the `metrics.k8s.io` API group.

D. Only `ClusterRoleBindings` can authorize `kubectl logs`.

Question 19: Verifying Permissions with `kubectl auth can-i`

As a cluster administrator, you just applied a complex set of `Roles` and `RoleBindings` for a ServiceAccount named `monitoring-sa` in the `observability` namespace. Which command lets you verify directly from your admin terminal whether `monitoring-sa` is allowed to list `pods` in the `prod` namespace?

A. `kubectl verify rbac monitoring-sa --namespace=prod --resource=pods`

B. `kubectl auth can-i list pods -n prod --as=system:serviceaccount:observability:monitoring-sa`

C. `kubectl describe serviceaccount monitoring-sa -n observability --show-permissions`

D. `kubectl get rolebindings --all-namespaces | grep monitoring-sa`

Question 20: Service Routing Failure Due to Label Mismatch

You deploy a `Deployment` and a `Service`, both in the `default` namespace. All 3 Pods are `1/1 Running`, and you can access the application directly using `kubectl port-forward pod/web-7f8b9-abcde 8080:80`. However, curling the Service ClusterIP times out. You check the configurations:

```
# Pod Labels:
metadata:
  labels:
    app: payment-service
    tier: backend
    version: v1.2

# Service Selector:
spec:
  selector:
    app: payment-service
    tier: backend
    env: prod
```

Why is the Service failing to route traffic to the Pods?

A. A Service selector cannot contain more than two labels.

B. Label selectors use logical **AND** matching; because the Service selector requires `env: prod` and the Pods do not have that label, the Service's `Endpoints` list is completely empty.

C. The Pods have an extra label (`version: v1.2`) not present in the Service selector, which causes the Service to reject them.

D. `kubectl port-forward` locks the container port so the Service cannot use it.

Question 21: Set-Based Label Selectors in Practice

Which `kubectl` command uses a valid **set-based** label selector to list all Pods that reside in either the `staging` or `prod` environment AND do **not** belong to the `legacy` tier?

A. `kubectl get pods -l 'environment in (staging, prod), tier notin (legacy)'`

B. `kubectl get pods --selector='environment == staging || prod && tier != legacy'`

C. `kubectl get pods --annotations='environment=staging,prod'`

D. `kubectl get pods -l 'environment=[staging,prod], !tier=legacy'`

Question 22: Labels vs. Annotations Misconception

An architect wants to attach the following metadata to a Deployment's Pods: 1) `tier: api` (used by NetworkPolicies and Services to target the Pods). 2) `build.company.io/commit-message: "Fix issue #402: handle null pointer in /login?redirect=true&mode=fast"` (used by an external dashboard for human auditing). How should these two pieces of metadata be stored, and why?

A. Store both in `metadata.labels` so they can both be queried by Services.

B. Store both in `metadata.annotations` because labels are deprecated in Kubernetes v1.

C. Store `tier: api` in `metadata.labels` (so selectors can query it) and store the commit message in `metadata.annotations` (because annotations allow arbitrary special characters/lengths up to 256 KB and are not used for object selection).

D. Store `tier: api` in `metadata.annotations` and the commit message in `metadata.finalizers`.

Question 23: Resource Stuck in `Terminating` State (`Finalizers`)

You attempt to delete a custom database object `db-backup-0` in the `legacy-app` namespace. After 15 minutes, `kubectl get backup db-backup-0 -n legacy-app -o yaml` shows the resource still exists with the following metadata:

```
metadata:
  deletionTimestamp: "2026-10-11T15:04:05Z"
  finalizers:
  - storage.company.io/cleanup-protection
  name: db-backup-0
  namespace: legacy-app
```

Why has Kubernetes not removed this object from `etcd`?

A. The object has an active `LimitRange` lock that prevents deletion until the worker node is rebooted.

B. When an object has entries in `metadata.finalizers`, deletion sets `deletionTimestamp` and places the object in a terminating state; Kubernetes waits until the responsible controller finishes its pre-delete cleanup logic and removes the finalizer entry from the array.

C. A `PodDisruptionBudget` is blocking the deletion because `deletionTimestamp` is set.

D. Objects with `finalizers` can only be deleted using `etcdctl` directly on the control plane node.

Question 24: Resolving a Stuck Finalizer Safely

In the scenario from Question 23, you discover that the custom controller responsible for `storage.company.i` `protection` was uninstalled from the cluster last week, and you have already manually verified that the underlying S3 storage was cleaned up. How can you force Kubernetes to complete the deletion of `db-backup-0`?

A. Run `kubectl delete backup db-backup-0 --grace-period=0 --force`; this automatically bypasses all finalizers in `etcd`.

B. Use `kubectl patch` or `kubectl edit` on `db-backup-0` to set `metadata.finalizers` to `null` (or an empty array `[]`), which immediately allows the API server to garbage-collect the object from `etcd`.

C. Restart the `kube-scheduler` pod on the control plane.

D. Scale the namespace replicas to 0.

2 Part 2: Pod Concepts, Architecture, Scheduling, Probes & Lifecycle (Q25–Q42)

Question 25: Why Kubernetes Manages Pods Instead of Containers Directly

An application consists of a main web server container and a helper log-shipping agent container that must read log files from a shared local directory and communicate over loopback with low latency. Why does Kubernetes group these into a single `Pod` rather than managing them as two separate standalone container deployments?

A. Because Linux kernels cannot run more than one container per virtual machine without a Pod wrapper.

B. A Pod provides a higher-level cohesive unit of deployment that guarantees co-scheduling onto the exact same physical node and shares the network namespace (IP/localhost) and Pod volumes between tightly coupled containers.

C. Putting multiple containers in a Pod merges their container images into a single unified root filesystem image automatically.

D. Kubernetes cannot assign CPU or memory limits to a single container unless there are at least two containers in a Pod.

Question 26: Multi-Container Pod Port Conflict Trace

You deploy a Pod containing two containers: `app-v1` (listening on port `8080`) and `app-v2` (also configured to listen on port `8080`). The Pod enters `CrashLoopBackOff`. Checking `kubectl logs my-pod -c app-v2` shows:

```
Error: listen EADDRINUSE: address already in use 0.0.0.0:8080
```

Why did this error occur?

A. Both containers share the exact same Linux **Network Namespace** (single IP address and port space); therefore, two containers inside the same Pod cannot bind to the same port number.

B. Docker/containerd assigns the host node's physical IP to every container by default.

C. Kubernetes Services do not allow port `8080` to be used by more than one container across the entire cluster.

D. The two containers are sharing the same Mount Namespace, which locks the socket file.

Question 27: Container Filesystem Isolation Misconception

A developer creates a Pod with two containers: `writer` and `reader`. No `volumes` or `volumeMounts` are defined in the Pod YAML. The `writer` container writes a file to `/tmp/report.json`. When the `reader` container tries to open `/tmp/report.json`, it fails with `FileNotFoundError: [Errno 2] No such file or directory: '/tmp/report.json'`. Why?

A. The `/tmp` directory is wiped by `kubelet` every 1 second.

B. While containers in a Pod share Network, IPC, and UTS namespaces, they **do not share the Mount Namespace (filesystem)**; each container has its own isolated root filesystem unless a Pod `Volume` (such as `emptyDir`) is explicitly mounted into both containers.

C. You must set `shareProcessNamespace: true` to share files between containers.

D. Containers in a Pod are scheduled onto different worker nodes by default.

Question 28: Sharing the PID Namespace Between Containers

You are running a minimal application container alongside a troubleshooting sidecar container in the same Pod. From inside the sidecar container, you want to run `ps aux` to inspect the running processes of the main application container and send it a `SIGHUP` signal to reload its configuration. By default, `ps aux` inside the sidecar only shows the sidecar's own processes. Which Pod spec setting enables the sidecar to see the main container's processes?

A. `hostNetwork: true`

B. `shareProcessNamespace: true`

C. `dnsPolicy: ClusterFirstWithHostNet`

D. `securityContext.privileged: false`

Question 29: Init Container Execution & Failure Behavior

A Pod has two `initContainers` (`init-db-check` and `init-schema-migrate`) and two regular application `containers` (`web` and `metrics-exporter`). When the Pod is scheduled, `kubectl get pods` shows:

```
NAME          READY   STATUS                  RESTARTS   AGE
backend-pod   0/2     Init:CrashLoopBackOff   4          2m
```

Which statement accurately describes the execution order and current state of this Pod?

A. All four containers (both init and regular containers) started simultaneously in parallel, and one of the init containers crashed.

B. Init containers execute strictly in sequential order (`init-db-check` must exit with code 0 before `init-schema-migrate` starts), and neither `web` nor `metrics-exporter` has started yet because an init container is failing.

C. The regular containers `web` and `metrics-exporter` run first, and `initContainers` only run when the Pod is being terminated.

D. Kubernetes will automatically skip the failing init container after 5 restarts and start the main `web` container.

Question 30: Debugging a Distroless Container with Ephemeral Containers

A production Pod running a Go binary packaged in a `gcr.io/distroless/static` image is experiencing intermittent network latency. You try to run `kubectl exec -it prod-api-pod -- /bin/sh`, which fails with:

```
OCI runtime exec failed: exec failed: unable to start container process:
exec: "/bin/sh": stat /bin/sh: no such file or directory
```

How can you inspect the network and processes of this running Pod without restarting or modifying the Deployment YAML?

A. Inject an **Ephemeral Container** containing debugging tools (e.g., `busybox` or `netshoot`) into the running Pod using `kubectl debug -it prod-api-pod --image=nicolaka/netshoot --target=prod-api`.

B. Edit the running Pod's `initContainers` array in place using `kubectl edit pod` to install `bash`.

C. Use `kubectl port-forward` to open an SSH shell on port 22.

D. It is impossible to debug distroless containers in Kubernetes without rebuilding the Docker image.

Question 31: Static Pod Deletion Mystery

On a kubeadm cluster, a junior admin notices a Pod named `kube-apiserver-master-01` in the `kube-system` namespace. Trying to test chaos engineering, they run `kubectl delete pod kube-apiserver-master-01 -n kube-system`. Seconds later, the exact same Pod reappears even though there is no Deployment, ReplicaSet, or DaemonSet managing it. Why?

A. It is a **Static Pod** managed directly by the `kubelet` daemon on `master-01` via a manifest file in `/etc/kubernetes/manifests`; deleting the mirror Pod via `kubectl` only deletes the API server representation, and `kubelet` immediately recreates it unless the local file on the node is removed.

B. The `etcd` database has a built-in CronJob that recreates control plane Pods every 5 seconds.

C. Pods in the `kube-system` namespace are immutable and ignore `DELETE` HTTP requests.

D. A `Finalizer` on the Pod cloned it before termination.

Question 32: Memory Limit Exceeded (`OOMKilled` / Exit Code 137) vs. CPU Limit

You configure a Java microservice container with the following resource constraints:

```
resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "500m"
    memory: "1Gi"
```

During peak traffic, the container tries to use `800m` of CPU and allocate `1.2Gi` of RAM. What happens to the container's CPU and Memory execution?

A. Both CPU and Memory spikes cause the container to be immediately killed with Exit Code 1.

B. CPU is a compressible resource and gets **throttled** at `500m` (slowing down processing without killing the container), whereas Memory is non-compressible and exceeding `1Gi` triggers the Linux kernel OOM killer to terminate the container with `OOMKilled` **(Exit Code 137)**.

C. Memory is swapped to the API server's disk, while exceeding the CPU limit kills the container with `CPUKilled`.

D. Kubernetes automatically scales up `limits.memory` to `2Gi` if the node has free RAM.

Question 33: Pod Quality of Service (QoS) Classes

When a worker node runs out of memory (Node-pressure Eviction), `kubelet` ranks running Pods by their Quality of Service (QoS) class ('BestEffort', 'Burstable', 'Guaranteed') to decide which Pods to evict first (assuming equal Pod priority). How must you configure a Pod so that Kubernetes assigns it the highest protection class, `Guaranteed`, and in what order are QoS classes evicted?

A. Do not specify any `requests` or `limits` on any container; eviction order is `Guaranteed` first, then `Burstable`, then `BestEffort`.

B. Every container in the Pod must have both `memory` and `cpu` `requests` and `limits` set, and for each resource, `requests` must strictly equal `limits`; eviction order under memory pressure is `BestEffort` first, then `Burstable`, and `Guaranteed` last.

C. Add the annotation `qos.kubernetes.io/class: Guaranteed` to the Pod metadata.

D. Set `limits` to be at least double the `requests` for every container.

Question 34: Liveness vs. Readiness Probe Cascading Failure

A microservice connects to an external shared Redis cache. The developer configures the Pod's `livenessProbe` to call an HTTP endpoint `/healthz` that pings the external Redis server. One afternoon, the Redis server experiences a temporary 30-second network glitch. Suddenly, all 50 Pods of the microservice restart simultaneously in a `CrashLoopBackOff` storm, bringing down the entire platform even for endpoints that do not use Redis. What was the architectural mistake?

A. A `livenessProbe` failure causes `kubelet` to kill and restart the container; checking external dependencies in a `livenessProbe` causes cascading restart storms when a downstream dependency flickers. Temporary downstream unavailability should be handled gracefully or isolated to a `readinessProbe` (which only removes the Pod from Service endpoints without restarting it).

B. A `readinessProbe` restarts the container, whereas a `livenessProbe` only removes the Pod IP from the Service.

C. Probes can only use `tcpSocket`, not `httpGet`.

D. `livenessProbe` only runs once when the Pod is first created.

Question 35: Protecting Slow-Starting Legacy Applications (`startupProbe`)

You are migrating a legacy monolith to Kubernetes. Normally, the application responds to health checks within 2 seconds, so you want a `livenessProbe` with `periodSeconds: 5` and `failureThreshold:`

3 (detecting deadlocks within 15 seconds). However, on initial startup, the monolith takes anywhere from 40 seconds to 3 minutes to warm up its internal caches. Without making deadlock detection sluggish during normal runtime, how should you prevent `kubelet` from killing the container before it finishes starting up?

A. Set `periodSeconds: 300` on the `livenessProbe`.

B. Configure a **`startupProbe`** (e.g., `failureThreshold: 30`, `periodSeconds: 10`), which disables the `livenessProbe` and `readinessProbe` until the `startupProbe` succeeds once.

C. Remove the `livenessProbe` and use a `Finalizer` instead.

D. Set `restartPolicy: Never` on the Deployment.

Question 36: Pod Stuck in `Pending` State — Scheduler Event Trace

You deploy a new machine learning Pod, and it remains stuck in the `Pending` state. Running `kubectl describe pod ml-worker` shows the following event from `default-scheduler`:

```
Warning  FailedScheduling  30s  default-scheduler  0/4 nodes are available:
1 Insufficient memory, 1 node(s) didn't match Pod's node affinity/selector,
2 node(s) had untolerated taint {dedicated: gpu}. preemption: 0/4 nodes are
available: 1 No preemption victims found for incoming pod, 3 Preemption is
not helpful for scheduling.
```

Based on this trace, why was the Pod not scheduled onto the 2 GPU nodes?

A. The 2 GPU nodes are out of disk space.

B. The 2 GPU nodes have a taint `dedicated=gpu:NoSchedule` (or `NoExecute`), and the `ml-worker` Pod spec is missing a matching `tolerations` entry for `key: "dedicated", value: "gpu"`.

C. The `kubelet` service is stopped on the 2 GPU nodes.

D. The container image failed to download from Docker Hub.

Question 37: Taints & Tolerations vs. Node Affinity Misconception

You have a 10-node cluster where 2 nodes have GPUs. You apply a taint `sku=gpu:NoSchedule` to the 2 GPU nodes, and you add a matching `toleration` for `sku=gpu:NoSchedule` to your AI Pod manifest. When you deploy the AI Pod, it gets scheduled onto one of the 8 regular non-GPU CPU nodes and fails because no GPU device is present! Why did the AI Pod land on a regular CPU node?

A. A `Toleration` only **permits** (allows) a Pod to be scheduled onto a tainted node; it does **not mandate** that the Pod must only run on that node. To force the Pod onto the GPU nodes, you must combine the `toleration` with a `nodeSelector` or `nodeAffinity` targeting the GPU nodes.

B. Taints only work on control plane nodes.

C. Adding a toleration automatically prevents a Pod from running on tainted nodes.

D. `NoSchedule` taints are ignored by `kube-scheduler` unless `PodAntiAffinity` is enabled.

Question 38: High Availability Across Nodes with `podAntiAffinity`

You have a 3-replica `Deployment` for a critical payment gateway in a 3-node cluster. During a routine deployment, you notice that `kube-scheduler` placed all 3 replicas onto `worker-01`. If `worker-01` crashes, the entire service goes down. Which Pod feature ensures that the scheduler refuses to place two replicas of this payment gateway on the same node?

A. `nodeSelector` with `kubernetes.io/hostname: worker-01`.

B. `podAntiAffinity` using `requiredDuringSchedulingIgnoredDuringExecution` with `topologyKey: "kubernetes.io/hostname"` matching the payment gateway's pod label.

C. `shareProcessNamespace: false`.

D. A `LimitRange` with `maxPodsPerNode: 1`.

Question 39: SecurityContext `RunContainerError` (`runAsNonRoot`)

A company security policy requires all Pods to include the following `securityContext`:

```
securityContext:
  runAsNonRoot: true
```

A developer deploys an official `nginx:latest` container with this setting, and the Pod fails to start with:

```
Error: container has runAsNonRoot and image will run as root (pod: "web-pod_default",
container: "nginx")
```

Why did this happen, and how can it be resolved?

A. The `nginx:latest` image Dockerfile defaults to `USER root` (UID 0); when `runAsNonRoot: true` is set, `kubelet` refuses to start the container unless the image specifies a numeric non-zero UID or you explicitly specify a non-zero `runAsUser` (e.g., `runAsUser: 101`) and use an unprivileged image/port.

B. `runAsNonRoot: true` is only valid on Windows worker nodes.

C. You must add `privileged: true` alongside `runAsNonRoot: true`.

D. The Pod needs a `ClusterRoleBinding` to run as non-root.

Question 40: Pod SecurityContext vs. Container SecurityContext Precedence

A Pod manifest defines both a Pod-level `securityContext` and a Container-level `securityContext`:

```
spec:
  securityContext:
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: app
    image: busybox
    securityContext:
      runAsUser: 3000
```

When the `app` container starts, which UID does its process run as?

A. UID `0` (root), because conflicting `runAsUser` values cancel each other out.

B. UID `1000`, because Pod-level settings always override Container-level settings.

C. UID `3000`, because Container-level `securityContext` settings override Pod-level `securityContext` settings for overlapping fields on that specific container.

D. UID `2000`, because `fsGroup` overrides `runAsUser`.

Question 41: Graceful Shutdown & `preStop` Lifecycle Hook

During a rolling update of a high-traffic web application, users occasionally experience `502 Bad Gateway` errors for 1–2 seconds when old Pods terminate. This happens because `kube-proxy` / Ingress controllers take a moment to update iptables/IPVS rules after the Pod enters `Terminating`, while the container inside the Pod immediately shuts down upon receiving `SIGTERM`. Which Pod feature is commonly used to delay container termination by a few seconds or trigger a graceful drain command before `SIGTERM` is sent?

A. An `initContainer` with a `sleep 10` command.

B. A container `lifecycle.preStop` hook (e.g., running `sleep 5` or `nginx -s quit`), which executes synchronously before `kubelet` sends the `SIGTERM` signal to the container process.

C. A `startupProbe` with `initialDelaySeconds: 10`.

D. Setting `restartPolicy: Never`.

Question 42: Understanding the `Unknown` Pod Lifecycle Phase

While monitoring a cluster, you see several Pods running on `worker-04` transition their `STATUS` to `Unknown` (or stay stuck when the node becomes `NotReady`). What causes a Pod's lifecycle phase to be reported as `Unknown`?

A. The container inside the Pod exited with an unrecognized exit code between 2 and 125.

B. The Pod was created without any labels or annotations.

C. For some reason (such as a network partition or `kubelet` daemon crash on the worker node), the Control Plane cannot communicate with the `kubelet` on the Pod's host node to determine the current state of the Pod.

D. The Pod is waiting for a `PersistentVolumeClaim` to bind.

3 Part 3: Workload Management & Controllers (Q43–Q60)

Question 43: ReplicaSet Label Selector Adoption Quirk

A `ReplicaSet` named `frontend-rs` is currently running in the `default` namespace with `replicas: 3` and selector `app=frontend`. All 3 Pods are healthy and `Running`. A developer wants to test a debug container and manually creates a standalone Pod using `kubectl run debug-pod --image=nginx -l app=frontend`. Instantly, `debug-pod` is terminated and disappears! Why did Kubernetes immediately terminate the standalone Pod?

A. `kubectl run` creates a Job that exits immediately by default.

B. A `ReplicaSet` continuously watches all Pods matching its label selector (`app=frontend`); when `debug-pod` was created with that label and no `ownerReference`, the ReplicaSet adopted it, saw 4 Pods running instead of the desired `replicas: 3`, and immediately terminated the extra Pod.

C. Two Pods cannot use the `nginx` image in the same namespace.

D. The `ResourceQuota` deleted the Pod.

Question 44: Why Deployments Are Preferred Over Direct ReplicaSets

Why do Kubernetes best practices recommend creating a `Deployment` rather than creating and managing a `ReplicaSet` directly for stateless applications?

A. A `ReplicaSet` can only run a maximum of 2 Pod replicas.

B. A `Deployment` manages `ReplicaSets` declaratively on your behalf, providing controlled server-side **Rolling Updates** (creating a new ReplicaSet and gradually scaling it up while scaling the old ReplicaSet down), revision history tracking, pause/resume capabilities, and instant **Rollbacks**.

C. Pods created by a `ReplicaSet` cannot be exposed via a Kubernetes `Service`.

D. `Deployments` do not store their state in `etcd`, making them faster than `ReplicaSets`.

Question 45: Immutable Deployment Selector Error Trace

You have an existing Deployment named `checkout-api`. You edit the Deployment manifest to add a new label `release: v2` under `spec.selector.matchLabels` and `spec.template.metadata.labels`, then run `kubectl apply -f deployment.yaml`. You receive the following error trace:

```
The Deployment "checkout-api" is invalid: spec.selector: Invalid value:
v1.LabelSelector{MatchLabels:map[string]string{"app":"checkout","release":"v2"}}:
field is immutable
```

Why does Kubernetes reject this change, and how can you update the Pod labels?

A. `spec.selector` on a Deployment is **immutable** after creation because changing it would orphan existing ReplicaSets and Pods; you can add new labels to `spec.template.metadata.labels` (as long as the original selector labels remain intact), or you must delete and recreate the Deployment if `spec.selector` itself must change.

B. You can only change `spec.selector` when the Deployment is scaled to 0 replicas.

C. Labels cannot contain numbers like `v2`.

D. You must use `kubectl rollout restart` before editing `spec.selector`.

Question 46: RollingUpdate `maxSurge` and `maxUnavailable` Calculation

A Deployment has `replicas: 10` and uses the `RollingUpdate` strategy configured with:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 20%
    maxUnavailable: 10%
```

During an image update rollout, what are the **maximum total number of Pods** that can exist at once and the **minimum number of available Ready Pods** guaranteed during the update?

A. Maximum 20 Pods total; minimum 1 Pod available.

B. Maximum 12 Pods total (`10 + 20%`); minimum 9 available Ready Pods (`10 - 10%`).

C. Maximum 10 Pods total; minimum 10 available Ready Pods.

D. Maximum 11 Pods total; minimum 8 available Ready Pods.

Question 47: Rolling Back a Broken Deployment Revision

After updating the `payment-gateway` Deployment to image `payment:v2.4.0`, the new Pods enter `CrashLoopBackOff`. Thanks to your `readinessProbe` and `RollingUpdate` strategy, the old ReplicaSet Pods are still serving traffic, while the rollout is stalled. Which command immediately rolls the Deployment back to the previous working revision?

A. `kubectl revert deployment payment-gateway --previous`

B. `kubectl rollout undo deployment/payment-gateway`

C. `kubectl delete replicaset --all`

D. `kubectl reset deployment payment-gateway --hard`

Question 48: StatefulSet Network Identity & Headless Service

You deploy a 3-replica `StatefulSet` named `mysql` governed by a Headless Service (`clusterIP: None`) named `mysql-hl` in the `db` namespace. Unlike a Deployment where Pods get random hash suffixes (e.g., `mysql-7d8f9-x2b`), what are the Pod names and stable DNS records assigned to the StatefulSet Pods?

A. Pods are named `mysql-1`, `mysql-2`, `mysql-3` and share a single DNS A record `mysql.db.pod`.

B. Pods are named `mysql-0`, `mysql-1`, `mysql-2`, and each Pod receives a deterministic, stable DNS domain name in the format `mysql-0.mysql-hl.db.svc.cluster.local`, `mysql-1.mysql-hl.db.svc.cluster.local`, etc.

C. Pods are named after the worker nodes they run on (`mysql-worker-01`).

D. A Headless Service assigns an external AWS Elastic IP to each Pod.

Question 49: StatefulSet Ordered Startup Blockage

You create a `StatefulSet` named `kafka` with `replicas: 3` (using the default `podManagementPolicy: OrderedReady`). You notice that `kafka-0` is created and running, `kafka-1` is stuck in `0/1 Running` (its `readinessProbe` is failing), and `kafka-2` has not even been created yet! Why has `kafka-2` not been created?

A. StatefulSets can only run a maximum of 2 replicas unless a `LimitRange` is added.

B. With `OrderedReady` pod management, a StatefulSet creates Pods strictly sequentially from `0` to `N-1`; it waits until predecessor Pod `kafka-1` is completely **Running and Ready** before it will create `kafka-2`.

C. `kafka-2` requires a separate `StorageClass` from `kafka-1`.

D. The Headless Service ran out of ClusterIPs.

Question 50: StatefulSet Scale-Down & PVC Retention Behavior

A `StatefulSet` named `redis-cluster` with `replicas: 3` uses `volumeClaimTemplates` to dynamically provision 3 PersistentVolumeClaims (`data-redis-cluster-0`, `data-redis-cluster-1`, `data-redis-cluster-2`). You scale the StatefulSet down to `replicas: 1`, and later scale it back up to `replicas: 3`. What happens to the PVCs `data-redis-cluster-1` and `data-redis-cluster-2` during this process?

A. Scaling down a StatefulSet immediately deletes the associated PVCs and wipes the underlying PersistentVolumes.

B. To protect stateful data from accidental loss, scaling down or deleting a StatefulSet **does NOT delete** its associated PVCs; when scaled back up to 3, the newly created `redis-cluster-1` and `redis-cluster-2` Pods automatically reattach to their existing PVCs (`data-redis-cluster-1` and `data-redis-cluster-2`) with all previous data intact.

C. Scaling back up creates brand-new PVCs named `data-redis-cluster-1-new` and leaves the old PVCs in a `Failed` state.

D. StatefulSets cannot be scaled down once created.

Question 51: StatefulSet Pod Stuck on a Dead Node

Worker node `worker-03` suffers a permanent motherboard failure and goes offline (`NotReady`). That node was running one stateless `Deployment` Pod and one `StatefulSet` Pod (`postgres-0`). After 5 minutes (the default pod eviction timeout), a new Deployment Pod is running on `worker-01`, but `postgres-0` remains stuck in `Terminating` on the dead `worker-03` node, and no replacement `postgres-0` Pod is created on a healthy node. Why does Kubernetes refuse to automatically recreate `postgres-0` on another node while `worker-03` is unreachable?

A. StatefulSets do not support self-healing under any circumstances.

B. Because StatefulSet Pods maintain a unique identity and write to dedicated storage, the Control Plane cannot confirm whether `postgres-0` is truly dead on `worker-03` or merely network-partitioned; starting a second `postgres-0` would cause a **split-brain / data corruption** scenario. The admin must confirm the node is dead/fenced before force-deleting the Pod.

C. `postgres-0` is a Static Pod.

D. A `ReplicaSet` is blocking the StatefulSet controller.

Question 52: DaemonSet Use Cases & Node Scaling Behavior

Your platform engineering team needs to deploy a Prometheus `node-exporter` agent and a `Fluent-bit` log collector so that every worker node currently in the cluster—and any new worker node automatically added by the Cluster Autoscaler in the future—runs exactly one instance of each agent. Which Workload Controller is specifically designed for this?

A. A `Deployment` with `replicas` equal to the current number of nodes.

B. A **`DaemonSet`**, which automatically ensures that every eligible node in the cluster runs a copy of the Pod, placing Pods on newly joined nodes and garbage-collecting them when nodes are removed.

C. A `StatefulSet` with `podManagementPolicy: Parallel`.

D. A `CronJob` running every 1 minute.

Question 53: Restricting a DaemonSet to Specific Nodes

By default, a `DaemonSet` schedules its Pod onto every worker node in the cluster. However, you want a specialized storage driver DaemonSet (`nvme-driver-ds`) to run **only** on worker nodes that have physical NVMe drives (labeled with `storage-tier=nvme`). How do you configure this?

A. Set `replicas: 2` inside the DaemonSet spec.

B. Specify a `nodeSelector` (e.g., `storage-tier: nvme`) or `nodeAffinity` inside the DaemonSet's `spec.template.spec`.

C. Create a `PodDisruptionBudget` matching `storage-tier: nvme`.

D. DaemonSets cannot be restricted to a subset of nodes; they must run on 100% of nodes.

Question 54: Job Manifest Validation Error (`restartPolicy`)

A developer writes a Kubernetes `Job` manifest to run a one-time database migration script. When they run `kubectl apply -f migration-job.yaml`, the API server rejects the manifest with:

```
The Job "db-migration" is invalid: spec.template.spec.restartPolicy:
Unsupported value: "Always": supported values: "OnFailure", "Never"
```

Why did this error occur, and what is the difference between the two valid options?

A. Pods defaults to `restartPolicy: Always`, which is invalid for a `Job` because a Job is meant to run to completion and terminate; in a Job, you must explicitly set `restartPolicy` to either `OnFailure` (kubelet restarts the failed container in-place on the same Pod/Node) or `Never` (the Job controller creates a brand-new Pod when a container fails).

B. Jobs only support `restartPolicy: Always` if `backoffLimit` is set to 0.

C. `restartPolicy` must be placed under `metadata.annotations` for a Job.

D. Only `CronJobs` allow `OnFailure`.

Question 55: Job Parallel Processing (`completions` vs. `parallelism`)

You have a batch processing `Job` configured with:

```
spec:
  completions: 12
  parallelism: 3
  backoffLimit: 4
```

How will Kubernetes execute this Job?

A. It will run 12 Pods simultaneously in parallel and repeat the entire batch 3 times.

B. It will run up to **3 Pods concurrently at any given time** until a total of **12 Pods have successfully completed** (exited with code 0), allowing at most 4 failures before marking the Job as failed.

C. It will run 1 Pod with 3 containers inside it for 12 minutes.

D. It will create 36 Pods immediately across the cluster.

Question 56: Automatic Cleanup of Finished Jobs (`ttlSecondsAfterFinished`)

Your cluster runs hundreds of automated one-off `Jobs` every day. Over time, thousands of completed Pods in the `Completed` (`Succeeded`) state accumulate in the namespace, cluttering `kubectl get pods` output and consuming API server / `etcd` object quota. Why doesn't Kubernetes delete finished Job Pods immediately upon completion, and what is the native way to clean them up automatically after 1 hour?

A. Kubernetes keeps finished Job Pods so you can inspect their logs (`kubectl logs`) and exit statuses; setting `ttlSecondsAfterFinished: 3600` in the Job spec instructs the TTL-after-finished controller to automatically delete the Job and its Pods 1 hour after completion.

B. Finished Jobs can only be deleted by restarting `etcd`.

C. Setting `backoffLimit: 3600` deletes the Job after 1 hour.

D. Adding a `livenessProbe` to a Job deletes it once it succeeds.

Question 57: CronJob `concurrencyPolicy` Preventative Tuning

You have a `CronJob` scheduled with `schedule: "*/5 * * * *"` (every 5 minutes) to generate financial reports. Occasionally, when data volume is heavy, a single report takes 8 minutes to finish. If a second Job starts while the first Job is still running, both Jobs lock the same database tables and corrupt the report. Which `CronJob` setting prevents a new Job run from starting if the previous run has not yet completed?

A. `concurrencyPolicy: Allow`

B. `concurrencyPolicy: Forbid`

C. `concurrencyPolicy: Replace`

D. `suspend: true`

Question 58: CronJob `Replace` vs. `Forbid` Behavior

In contrast to Question 57, suppose another `CronJob` syncs ephemeral cache data every 10 minutes. If a run gets stuck and is still running at the 10-minute mark when the next scheduled time arrives, you want Kubernetes to **terminate the currently running unfinished Job and start a fresh new Job instance**. Which setting accomplishes this?

A. `concurrencyPolicy: Replace`

B. `concurrencyPolicy: Forbid`

C. `startingDeadlineSeconds: 1`

D. `completions: 0`

Question 59: Port-Forward Debugging vs. Production Service Exposure

A new developer runs `kubectl port-forward pod/api-server-pod 8080:80` on their laptop and sees the application working at `http://localhost:8080`. They propose running `kubectl port-forward` in a background terminal on a jump server as the permanent production load balancer for external users. Why is this a poor practice compared to using a Kubernetes `Service`?

A. `kubectl port-forward` is a debugging utility that establishes a single multiplexed tunnel through the API server to a specific Pod; it does not load-balance across multiple Pod replicas cleanly, dies whenever the target Pod restarts or the client connection drops, and bottlenecks traffic through `kube-apiserver`.

B. `kubectl port-forward` automatically deletes the Pod when you press `CTRL+C`.

C. `kubectl port-forward` only supports UDP traffic, not TCP/HTTP.

D. Kubernetes Services cannot forward port 80.

Question 60: End-to-End Troubleshooting — `0/1 Running` Pod & Service `503`

You deploy a new `Deployment` (`replicas: 1`) and a `Service`. Running `kubectl get pods` shows:

```
NAME                              READY   STATUS    RESTARTS   AGE
catalog-api-5d89b7c64b-9xk2p   0/1     Running   0          4m
```

Notice that `STATUS` is `Running` and `RESTARTS` is `0`, yet `READY` is `0/1` and requests to the Service fail with `503 Service Unavailable`. Running `kubectl describe pod catalog-api-5d89b7c64b-9xk2p` shows:

```
Warning  Unhealthy  10s (x22 over 3m40s)  kubelet  Readiness probe failed:
Get "http://10.244.1.45:8080/ready": dial tcp 10.244.1.45:8080: connect: connection refused
```

You check the container logs and see: `Server listening on port 3000`. What is the exact chain of events happening here?

A. The container image failed to pull from the registry.

B. The application inside the container is running fine on port `3000` and there is no failing `livenessProbe` (so `RESTARTS` is `0`), but the `readinessProbe` is misconfigured to check port `8080`; consequently, `kubelet` marks the Pod as not Ready (`0/1`) and the Service controller removes the Pod IP from the Service's `Endpoints`.

C. A `ResourceQuota` is throttling the network interface of the Pod.

D. The `Finalizer` on the Service is blocking port `8080`.

4 Answer Key & Detailed Technical Explanations

Part 1: Section 8 Answers (Q1–Q24)

Q1. Answer: B
Explanation: In Kubernetes RBAC, every resource belongs to a specific API Group. While core resources like `pods`, `services`, `secrets`, and `configmaps` belong to the core API group (`apiGroups: [""]`), `deployments`, `replicasets`, `statefulsets`, and `daemonsets` belong to the `"apps"` API group (`apiGroups: ["apps"]`).

Q2. Answer: B
Explanation: A `CustomResourceDefinition` (CRD) only registers a new schema and API endpoint with `kube-apiserver` so custom objects can be validated and stored in `etcd`. To actually perform backend actions (like running an etcd backup and uploading to S3), a **Custom Controller** (the Operator pattern) must be running to watch the custom resource and reconcile its desired state.

Q3. Answer: C
Explanation: Object name uniqueness in Kubernetes is scoped by '(apiVersion/Kind, namespace, name)'. Thus, a `Pod` and a `Service` can share the exact same name in the same namespace. However, every object created in Kubernetes receives a distinct Universally Unique Identifier (`UID`) across the entire lifecycle of the cluster; deleting and recreating a Pod produces a brand-new `UID`.

Q4. Answer: B
Explanation: Kubernetes does not enforce a strict count limit on the number of objects separated by `---` in a manifest file, but there is a default payload size limit of approximately 1 MB (and an `etcd` storage limit of 1.5 MB per entry). Best practices recommend splitting large manifests into logical files for maintainability.

Q5. Answer: B
Explanation: When a Pod references a missing ConfigMap or Secret key via `configMapKeyRef` or `secretKeyRef`, `kubelet` fails container creation with `CreateContainerConfigError`. Adding `optional: true` inside the `configMapKeyRef` block allows the Pod to start even if the ConfigMap or the specific key does not exist.

Q6. Answer: B
Explanation: Normally, ConfigMaps and Secrets mounted as volumes are updated automatically via atomic symlink swaps after the kubelet sync period. However, a classic Kubernetes gotcha is that volume mounts using **`subPath`** bind-mount a single file directly and **never receive automatic live updates** when the ConfigMap or Secret changes.

Q7. Answer: C
Explanation: `ConfigMaps` and `Secrets` are strictly namespaced and can **only** be mounted or referenced as environment variables by Pods residing in the **exact same namespace**. In contrast, Kubernetes `Services` can be accessed across namespaces using their FQDN (`<service-name>.<namespace>.svc.cluster.local`).

Q8. Answer: B
Explanation: `PersistentVolumes` (PVs), `StorageClasses`, `Nodes`, `ClusterRoles`, `ClusterRoleBindings`, and `Namespaces` are cluster-scoped resources. Deleting a namespace deletes all namespaced resources inside it (including PVCs), but cluster-scoped objects remain intact.

Q9. Answer: B
Explanation: When an active `ResourceQuota` in a namespace restricts compute resources (`requests.cpu`, `limits.memory`, etc.), Kubernetes requires **every incoming Pod** in that

namespace to explicitly declare those resource fields in its spec—unless a `LimitRange` exists in the namespace to automatically inject default values (`defaultRequest` / `default`).

Q10. Answer: C
Explanation: `LimitRange` acts as an Admission Controller (both mutating to inject defaults and validating to check min/max bounds) when a Pod is created or updated. It does not retroactively terminate or modify Pods that are already running in the namespace.

Q11. Answer: B
Explanation: With `replicas: 2` and `minAvailable: 2`, the number of allowed voluntary disruptions is `2 - 2 = 0`. Consequently, `kubectl drain` (which uses the Eviction API) is blocked indefinitely. For a PDB to allow rolling node drains, `minAvailable` must be strictly less than the replica count (or `maxUnavailable` must be at least 1).

Q12. Answer: B
Explanation: `PodDisruptionBudgets` (PDBs) only protect applications against **voluntary disruptions** (actions initiated through the Kubernetes Eviction API such as `kubectl drain`, node upgrades, or cluster autoscaler scale-down). They cannot prevent **involuntary disruptions** like physical power outages, kernel panics, or hardware crashes.

Q13. Answer: B
Explanation: When a high-priority Pod cannot be scheduled due to insufficient node resources, `kube-scheduler` evaluates **Preemption** logic: if evicting lower-priority Pods on a node frees enough resources to satisfy the high-priority Pod's requests (and all other scheduling constraints), the scheduler gracefully terminates the lower-priority Pods.

Q14. Answer: A
Explanation: Base64 encoding in a Kubernetes `Secret` is merely an ASCII encoding format (easily decoded via `base64 -d`), **not encryption**. By default, Secrets are persisted in unencrypted plaintext in `etcd`. Production clusters should enable Encryption at Rest (`EncryptionConfiguration`) and strict RBAC.

Q15. Answer: B
Explanation: To authenticate with a private container registry, `kubelet` requires a Secret of type `kubernetes.io/dockerconfigjson` linked to the Pod via `spec.imagePullSecrets` (or attached to the Pod's `ServiceAccount`).

Q16. Answer: B
Explanation: By default, Kubernetes automatically mounts a ServiceAccount token into every Pod. If an application does not need to talk to the Kubernetes API, setting `automountServiceAccoun` `false` on the Pod spec or ServiceAccount follows the principle of least privilege.

Q17. Answer: A
Explanation: A major RBAC best practice is defining common permission sets once in a `ClusterRole` and binding that `ClusterRole` inside individual namespaces using a namespaced **`RoleBinding`**. When a `RoleBinding` references a `ClusterRole`, the permissions are strictly scoped to the namespace of the `RoleBinding`.

Q18. Answer: B
Explanation: Granting `get` or `create` on `["pods"]` only covers the Pod object metadata/spec itself. Viewing container logs requires `get` on the `pods/log` subresource, and executing commands inside a container requires `create`/`get` on the `pods/exec` subresource.

Q19. Answer: B
Explanation: The `kubectl auth can-i <verb> <resource> -n <namespace> --as=<user/sa>` command queries the API server's `SelfSubjectAccessReview` / `SubjectAccessReview` API and supports impersonation via `--as=system:serviceaccount:<ns>:<sa-name>`.

Q20. Answer: B
Explanation: For a Service to select a Pod, the Pod must possess **all** key-value pairs specified in the Service's `spec.selector` (logical AND). Extra labels on the Pod are fine, but missing a required selector label (`env: prod`) results in zero matching Endpoints.

Q21. Answer: A
Explanation: Set-based label selectors support three operators: `in`, `notin`, and key existence (`key` or `!key`). Thus, `'environment in (staging, prod), tier notin (legacy)'` is the exact valid syntax.

Q22. Answer: C
Explanation: **Labels** have strict character and length constraints (max 63 characters, alphanumeric with -, _, .) and are indexed for fast selector queries. **Annotations** are non-identifying metadata that cannot be queried by selectors, making them ideal for URLs, JSON blobs, Git commit messages, and tool metadata.

Q23. Answer: B
Explanation: **Finalizers** act as asynchronous pre-delete hooks. When you issue a `DELETE` request on an object with finalizers, Kubernetes sets `metadata.deletionTimestamp` (putting it into `Terminating`) and blocks actual removal from `etcd` until the controller removes its string from `metadata.finalizers`.

Q24. Answer: B
Explanation: Passing `--force --grace-period=0` does not bypass `metadata.finalizers` in `etcd`. When a controller is permanently gone and external resources are already cleaned up, patching or editing the object to empty the `metadata.finalizers` list allows the API server to immediately finalize deletion.

Part 2: Section 9 Answers (Q25–Q42)

Q25. Answer: B
Explanation: A Pod abstracts a "logical host" for one or more tightly coupled containers (such as a main app and a sidecar). It guarantees they are scheduled together on the same worker node and share the same network namespace (IP and localhost ports), IPC namespace, UTS hostname, and shared Pod volumes.

Q26. Answer: A
Explanation: Because all containers in a Pod share a single Linux **Network Namespace** (one IP address and one TCP/UDP port space), two containers inside the same Pod cannot listen on the exact same port (`8080`) without causing an `EADDRINUSE` port clash.

Q27. Answer: B
Explanation: A very common misconception is that containers in a Pod share their filesystem by default. In reality, each container has its own isolated **Mount Namespace** (root filesystem from its container image). To share files between containers in a Pod, you must define a Pod `Volume` (such as `emptyDir`) and mount it into both containers.

Q28. Answer: B
Explanation: By default, the PID namespace is isolated per container inside a Pod. Setting `shareProcessNamespace: true` in the Pod spec merges the PID namespace across all containers in the Pod, allowing a sidecar or debug container to view processes and send signals to the main application container.

Q29. Answer: B
Explanation: `initContainers` run **one at a time in sequential order** before any regular application containers start. Each init container must exit successfully (exit code 0) before

the next one begins. If an init container fails, Kubernetes restarts it according to the Pod's `restartPolicy`, blocking regular containers from starting.

Q30. Answer: A
Explanation: **Ephemeral Containers** (`kubectl debug`) allow administrators to attach a temporary troubleshooting container (with full shell and network utilities) directly into the namespaces of a live, running distroless Pod without restarting the Pod or modifying its image.

Q31. Answer: A
Explanation: Control plane components (`kube-apiserver`, `etcd`, `kube-scheduler`, `kube-controller-manager`) in kubeadm clusters run as **Static Pods** managed directly by the local `kubelet` reading YAML manifests from `/etc/kubernetes/manifests`. The API server only displays a read-only "mirror Pod"; deleting it via `kubectl` causes `kubelet` to immediately recreate it from the local file.

Q32. Answer: B
Explanation: CPU is a compressible resource: when a container hits `limits.cpu`, the Linux CFS scheduler throttles its CPU cycles without killing it. Memory is a non-compressible resource: when a container exceeds `limits.memory`, the kernel Out-Of-Memory killer terminates the process with `OOMKilled` (Exit Code 137 = $128 + 9$ `SIGKILL`).

Q33. Answer: B
Explanation: Kubernetes assigns a Pod the **`Guaranteed`** QoS class only if **every container** in the Pod specifies both CPU and Memory requests and limits, and `requests == limits` for each. Under node memory pressure, `kubelet` evicts `BestEffort` Pods first (no requests/limits), then `Burstable` Pods, and protects `Guaranteed` Pods until last.

Q34. Answer: A
Explanation: If a `livenessProbe` checks an external dependency (like a shared DB or Redis) and that dependency experiences a brief network hiccup, `kubelet` kills and restarts every Pod at the same time. Liveness probes should only verify internal process responsiveness/deadlocks, while `readinessProbes` control traffic routing without killing containers.

Q35. Answer: B
Explanation: A **`startupProbe`** is specifically designed for slow-starting containers. While the `startupProbe` is running, `livenessProbe` and `readinessProbe` checks are disabled. Once the `startupProbe` succeeds once, the fast-reacting `livenessProbe` takes over for the remainder of the container's lifecycle.

Q36. Answer: B
Explanation: The scheduler trace explicitly states `2 node(s) had untolerated taint {dedicated: gpu}`. Because those nodes have a taint, `kube-scheduler` will not place any Pod on them unless the Pod spec includes a matching `tolerations` entry.

Q37. Answer: A
Explanation: Taints and Tolerations work by **repulsion/permission**, not attraction. Adding a toleration allows the Pod to land on the GPU nodes, but does not prevent it from landing on untainted regular CPU nodes. To guarantee placement on GPU nodes, you must pair the `toleration` with `nodeSelector` or `nodeAffinity`.

Q38. Answer: B
Explanation: `podAntiAffinity` with `requiredDuringSchedulingIgnoredDuringExecution` and `topologyKey: "kubernetes.io/hostname"` instructs `kube-scheduler` never to schedule a Pod onto a node that is already running another Pod matching the same label selector.

Q39. Answer: A
Explanation: When `runAsNonRoot: true` is active, `kubelet` verifies that the container will not run as UID 0. Because standard `nginx:latest` defaults to root (`UID 0`), `kubelet` blocks container startup. You must either specify a non-zero numeric `runAsUser` in the `securityContext` or use a non-root image (such as `nginxinc/nginx-unprivileged`) with a non-zero numeric `USER`.

Q40. Answer: C
Explanation: When a field like `runAsUser` is defined at both the Pod level (`spec.securityContext`) and the Container level (`spec.containers[*].securityContext`), the **Container-level setting takes precedence** for that container, while Pod-level fields not overridden (like `fsGroup: 2000`) still apply.

Q41. Answer: B
Explanation: When a Pod is deleted, endpoint removal (propagating to `kube-proxy` and Ingress controllers) happens asynchronously in parallel with Pod termination (`preStop` hook $\rightarrow$ `SIGTERM`). Adding a short `preStop` hook (such as a brief sleep or graceful drain command) gives load balancers time to stop sending new connections before the container process stops accepting them.

Q42. Answer: C
Explanation: The `Unknown` phase indicates that the state of the Pod could not be obtained by the API server, typically because communication with the `kubelet` on the host node where the Pod is running has been lost.

Part 3: Section 10 Answers (Q43–Q60)

Q43. Answer: B
Explanation: Controllers in Kubernetes identify their Pods strictly via **Label Selectors**. If you create a bare Pod with labels matching an active `ReplicaSet`'s selector (and without an existing controller `ownerReference`), the ReplicaSet adopts the Pod, sees that the active count (4) exceeds `replicas: 3`, and immediately terminates one Pod to reconcile the state.

Q44. Answer: B
Explanation: A `ReplicaSet` only ensures a fixed number of identical Pods are running. A `Deployment` sits above `ReplicaSets` to orchestrate declarative updates—creating a new ReplicaSet when the Pod template changes, rolling traffic over seamlessly, recording rollout revisions, and allowing instant rollbacks.

Q45. Answer: A
Explanation: In a `Deployment` (as well as `StatefulSet` and `DaemonSet`), `spec.selector` is **immutable** once created. You can add additional labels to `spec.template.metadata.labels`, provided the original `spec.selector.matchLabels` remain present on the template.

Q46. Answer: B
Explanation: With `replicas: 10`, `maxSurge: 20%` allows at most $10 \times 0.20 = 2$ extra Pods above the desired count (maximum $10 + 2 = 12$ total Pods). `maxUnavailable: 10%` allows at most $10 \times 0.10 = 1$ Pod to be unavailable during the update (guaranteeing at least $10 - 1 = 9$ available Ready Pods).

Q47. Answer: B
Explanation: `kubectl rollout undo deployment/<name>` immediately reverts the Deployment's Pod template back to the previous revision (or a specific revision via `--to-revision=N`), scaling up the previous working ReplicaSet.

Q48. Answer: B
Explanation: `StatefulSets` assign deterministic, sticky ordinal indices starting from `0` (`mysql-0`, `mysql-1`, `mysql-2`). When paired with a Headless Service (`clusterIP: None`), CoreDNS creates individual DNS A/AAAA records for each Pod in the form `<pod-name>.<service-name>.<namespace>.svc.cluster.local`.

Q49. Answer: B
Explanation: By default (`podManagementPolicy: OrderedReady`), a `StatefulSet` enforces strict sequential startup: Pod `N` will not be deployed until Pod `N-1` is both `Running` and completely `Ready` (passing its `readinessProbe`). Because `kafka-1` is failing its readiness probe, `kafka-2` is blocked from creation.

Q50. Answer: B
Explanation: As a safety mechanism for stateful workloads, scaling down or deleting a `StatefulSet` **never deletes** the `PersistentVolumeClaims` created via `volumeClaimTemplates` (unless automatic PVC retention policy is explicitly configured to delete). When scaled back up, the new Pods reconnect to their matching ordinal PVCs.

Q51. Answer: B
Explanation: Because a `StatefulSet` Pod has a unique network identity and exclusive write access to its storage volume, Kubernetes refuses to automatically reschedule `postgres-0` onto another node while the original node is merely unreachable (`NotReady`). Doing so while the old container might still be running on a partitioned node could cause split-brain storage corruption.

Q52. Answer: B
Explanation: A `DaemonSet` ensures that all (or selected) nodes run a single copy of a Pod. As nodes are added to the cluster, DaemonSet Pods are automatically placed on them; as nodes are removed, those Pods are garbage-collected.

Q53. Answer: B
Explanation: You can restrict a `DaemonSet` to run only on a specific subset of nodes by configuring `nodeSelector` or `nodeAffinity` inside the DaemonSet's Pod template (`spec.template.spec`).

Q54. Answer: A
Explanation: Because a `Job` manages Pods that are meant to run a task to completion and exit, Kubernetes forbids `restartPolicy: Always` on Job Pods. You must explicitly specify either `OnFailure` (restarts container inside the same Pod) or `Never` (creates a new replacement Pod on failure).

Q55. Answer: B
Explanation: In a `Job`, `parallelism: 3` defines the maximum number of Pods allowed to run concurrently at one time, while `completions: 12` defines the total number of successful Pod completions required before the Job itself is marked complete.

Q56. Answer: A
Explanation: Completed Job Pods are retained by default so operators can inspect their output logs and exit codes. Configuring `ttlSecondsAfterFinished` enables automatic garbage collection of finished Jobs and their child Pods after the specified number of seconds.

Q57. Answer: B
Explanation: Setting `concurrencyPolicy: Forbid` on a `CronJob` guarantees that if the time arrives for a new Job run while the previous Job run is still active, the CronJob controller will **skip** the new run until the current run finishes.

Q58. Answer: A

Explanation: Setting `concurrencyPolicy: Replace` instructs the `CronJob` controller to cancel/delete the currently running unfinished Job and replace it with a newly scheduled Job instance.

Q59. Answer: A

Explanation: `kubectl port-forward` is strictly a developer/admin debugging utility that tunnels traffic from a local workstation through the API server to a single Pod. Production traffic should always use Kubernetes `Services` (and `Ingress` / `Gateway API`) for resilient load balancing across healthy Pod replicas.

Q60. Answer: B

Explanation: Because there is no failing `livenessProbe` and the main process is running normally on port `3000`, the container stays `Running` with `0` restarts. However, because the `readinessProbe` is checking the wrong port (`8080`), it fails continuously, keeping the Pod in `0/1 Ready` state and removing its IP from the Service's `Endpoints`, which causes the Service to return `503 Service Unavailable`.